

開始使用 App Engine (Python 3)

來源：<https://codelabs.developers.google.com/codelabs/cloud-app-engine-python3?hl=zh-tw>

步驟數：9

本教學課程將說明如何在 Google App Engine 中使用 Python 建構簡單的網頁應用程式。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 編寫網頁應用程式](#)
4. [4. 定義依附元件](#)
5. [5. 設定部署作業](#)
6. [6. 部署網頁應用程式](#)
7. [7. 測試網頁應用程式](#)
8. [8. 更新網頁應用程式](#)
9. [9. 恭喜！](#)

1. 總覽



Google App Engine 應用程式易於建立、維護及擴充，可因應流量和資料儲存空間需求變化。使用 App Engine 就不需要維護伺服器。只要上傳應用程式，就能開始使用。

在本程式碼研究室中，您將瞭解如何部署以 [Flask](#) 網路架構編寫的簡易 Python 網頁應用程式。雖然本範例使用 Flask，但您也可以使用其他網路架構，包括 [Django](#)、[Pyramid](#)、[Bottle](#) 和 [web.py](#)。

本教學課程改編自 <https://cloud.google.com/appengine/docs/standard/python3/quickstart>

課程內容

- 瞭解如何在 Google App Engine 上建立簡單的 Python 伺服器。
- 如何在不關閉伺服器的情況下更新程式碼。

軟硬體需求

- 熟悉 Python
- 熟悉標準 Linux 文字編輯器，例如 vim、emacs 或 nano

問卷調查

步驟 2 · 約 1 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots from the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top navigation bar, with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form. The 'Project name' field contains 'My Project 13475'. The 'Project ID' field contains 'inbound-analogy-389614', which is highlighted with a blue box. The 'Location' field is empty, and the 'BROWSE' button is visible. The 'CREATE' and 'CANCEL' buttons are at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

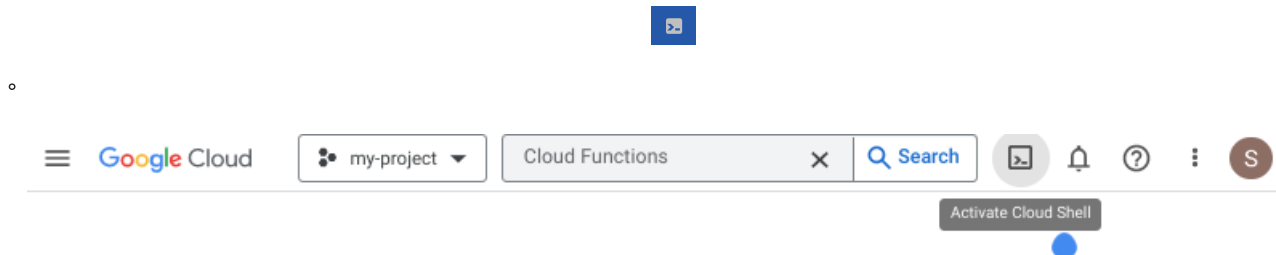
用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

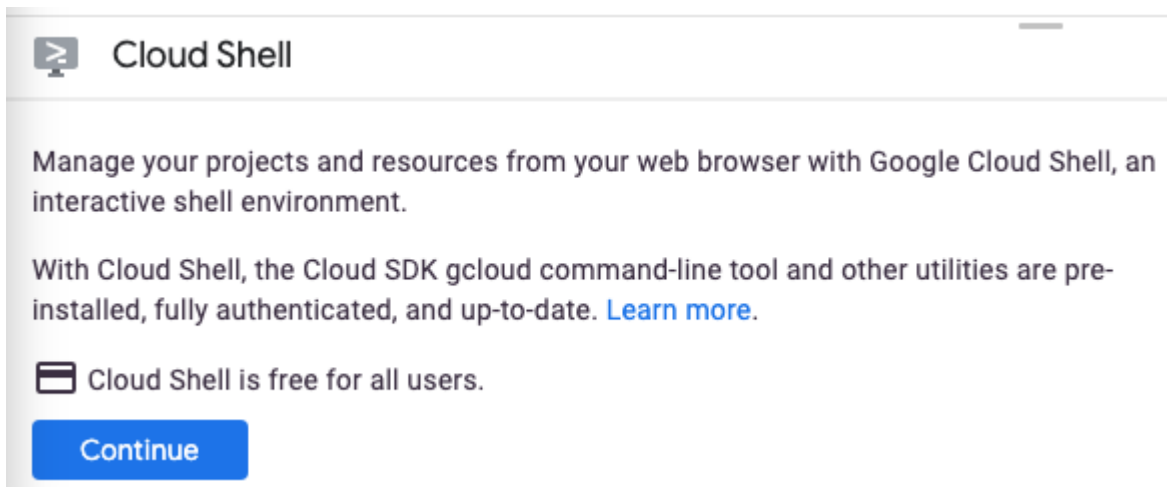
雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

啟用 Cloud Shell

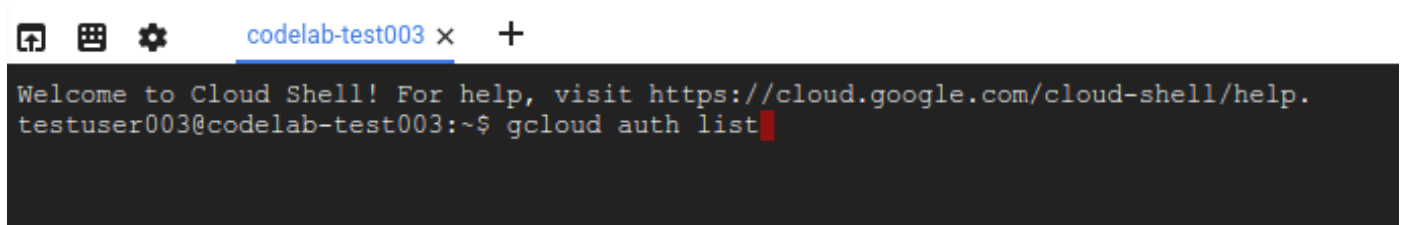
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。**第一次執行指令時**，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意：如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 1 分鐘

3. 編寫網頁應用程式

啟動 Cloud Shell 後，您可以透過指令列叫用 Cloud SDK `gcloud` 指令，或叫用虛擬機器執行個體提供的其他工具。您可以使用永久磁碟儲存空間中的 `$HOME` 目錄，儲存各專案和 Cloud Shell 工作階段的檔案。只有您能使用自己的 `$HOME` 目錄，其他使用者均無法存取。

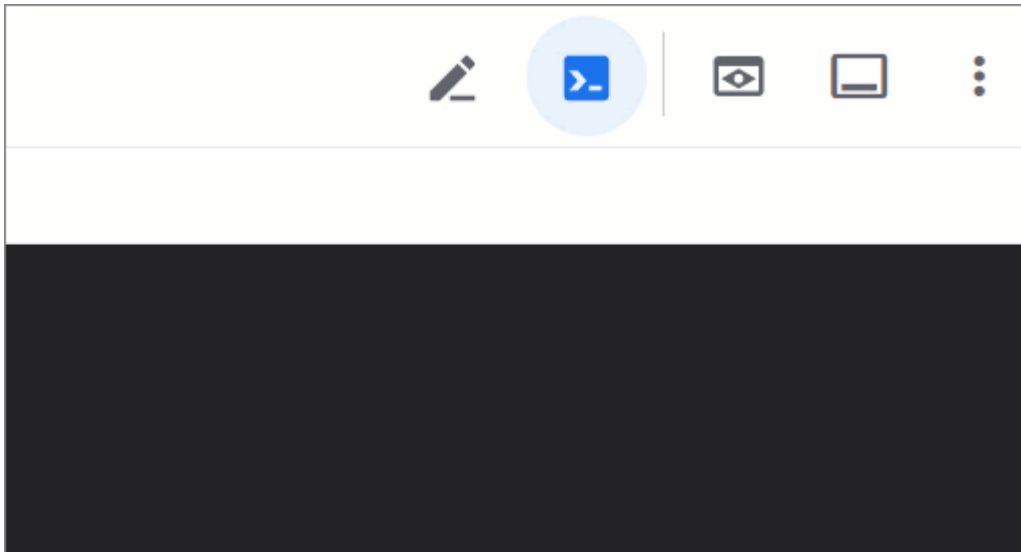
首先，請在應用程式的 `$HOME` 目錄中建立新資料夾：

```
mkdir ~/helloworld  
cd ~/helloworld
```

建立名為 `main.py` 的檔案：

```
touch main.py
```

使用偏好的指令列編輯器 (`nano`、`vim` 或 `emacs`) 編輯檔案，或點按 Cloud Shell 編輯器按鈕：



如要使用 Cloud Shell 編輯器直接編輯檔案，請使用下列指令：

```
cloudshell edit main.py
```

main.py

```
import flask

# If `entrypoint` is not defined in app.yaml, App Engine will look for an app
# called `app` in `main.py`.
app = flask.Flask(__name__)

@app.get("/")
def hello():
    """Return a friendly HTTP greeting."""
    return "Hello World!\n"

if __name__ == "__main__":
    # Used when running locally only. When deploying to Google App
    # Engine, a webserver process such as Gunicorn will serve the app. This
    # can be configured by adding an `entrypoint` to app.yaml.
    app.run(host="localhost", port=8080, debug=True)
```

這個網頁應用程式是簡單的網路服務，會以 `Hello World!` 訊息回應 HTTP GET 要求。

步驟 4 · 約 1 分鐘

4. 定義依附元件

如要指定網頁應用程式的依附元件，請返回終端機，在專案的根目錄中建立 `requirements.txt` 檔案，並指定要使用的 Flask 確切版本：

```
touch requirements.txt
```

如要使用 Cloud Shell 編輯器編輯檔案，請使用下列指令：

```
cloudshell edit requirements.txt
```

requirements.txt

```
# https://pypi.org/project/Flask
Flask==3.0.2
```

網頁應用程式的主要依附元件是 Flask。

步驟 5 · 約 1 分鐘

5. 設定部署作業

如要將網頁應用程式部署至 App Engine，您需要 `app.yaml` 檔案。這個設定檔會定義網路應用程式的 App Engine 設定。

在終端機中，於專案的根目錄建立並編輯 `app.yaml` 檔案：

```
touch app.yaml
```

如要使用 Cloud Shell 編輯器編輯檔案，請使用下列指令：

```
cloudshell edit app.yaml
```

app.yaml

```
runtime: python312
```

使用這個簡單的網路應用程式時，您只需要指定 Python 版本 (例如這裡的 `3.12`)。如果是較複雜的網路應用程式，您可以設定其他設定，例如資源調度、靜態檔案的處理常式、其他處理常式，以及環境變數和服務名稱等應用程式元素。如要進一步瞭解所有支援的項目並取得完整清單，請參閱 `app.yaml` [參考資料](#)。

步驟 6 · 約 1 分鐘

6. 部署網頁應用程式

這裡會省略幾個步驟。在標準開發週期中，您會先設定本機開發環境、在本機安裝相同的依附元件，並在本機測試應用程式，然後再進行部署。請參閱「[設定 Python 開發環境](#)」。

從終端機檢查目錄內容：

```
ls
```

您應該會有下列 3 個檔案：

```
app.yaml  main.py  requirements.txt
```

使用下列指令部署網頁應用程式：

```
gcloud app deploy
```

首次使用時，您需要選擇部署區域：

```
Please choose the region where you want your App Engine application
located:
```

```
[1] asia-east2
...
[7] australia-southeast1
[8] europe-west
[9] europe-west2
...
[12] northamerica-northeast1
[13] southamerica-east1
...
[19] us-west4
...
Please enter your numeric choice:
```

確認要啟動部署作業：

```
Creating App Engine application in project [PROJECT_ID] and region [REGION]....done.  
Services to deploy:
```

```
descriptor:      [~/helloworld/app.yaml]  
source:          [~/helloworld]  
target project:  [PROJECT_ID]  
target service:  [default]  
target version:  [YYYYMMDDtHHMMSS]  
target url:      [https://PROJECT_ID.REGION_ID.r.appspot.com]
```

```
Do you want to continue (Y/n)?
```

部署應用程式：

```
Beginning deployment of service [default]...  
Created .gcloudignore file. See `gcloud topic gcloudignore` for details.  
Uploading 3 files to Google Cloud Storage  
100%  
File upload done.  
Updating service [default]...done.  
Setting traffic split for service [default]...done.  
Deployed service [default] to [https://PROJECT_ID.REGION_ID.r.appspot.com]
```

您的網頁應用程式現在已準備好回應 https://PROJECT_ID.REGION_ID.r.appspot.com 上的 HTTP 要求。

步驟 7 · 約 1 分鐘

7. 測試網頁應用程式

您的網路應用程式已準備好回應 `https://PROJECT_ID.REGION_ID.r.appspot.com` 的 HTTP 要求。

首先，使用 `gcloud app describe` 指令擷取網頁應用程式主機名稱：

```
APPENGINE_HOSTNAME=$(gcloud app describe --format "value(defaultHostname)")
```

使用這個簡單的 HTTP GET 要求測試網路應用程式：

```
curl https://$APPENGINE_HOSTNAME
```

您應該會收到下列答案：

```
Hello World!
```

您可以透過 `example.com` 等自訂網域 (而非預設的 `appspot.com` 位址) 提供網頁應用程式。請參閱「[對應自訂網域](#)」。

摘要

在先前的步驟中，您設定了簡單的 Python 網頁應用程式，並在 App Engine 上執行及部署該應用程式。

步驟 8 · 約 1 分鐘

8. 更新網頁應用程式

在 `main.py` 檔案中變更 `hello()` 函式主體，即可修改網路應用程式。

如要使用 Cloud Shell 編輯器編輯檔案，請使用下列指令：

```
cloudshell edit main.py
```

main.py

```
import flask

# If `entrypoint` is not defined in app.yaml, App Engine will look for an app
# called `app` in `main.py`.
app = flask.Flask(__name__)

@app.get("/")
def hello():
    """Return a friendly HTTP greeting."""
    # return "Hello World!\n" # ← Replace this line
    who = flask.request.args.get("who", "World")
    return f"Hello {who}!\n"

if __name__ == "__main__":
    # Used when running locally only. When deploying to Google App
    # Engine, a webserver process such as Gunicorn will serve the app. This
    # can be configured by adding an `entrypoint` to app.yaml.
    app.run(host="localhost", port=8080, debug=True)
```

這裡使用 Flask 的 `request` 內容物件，處理 HTTP GET 要求中的選用參數。

從終端機重新部署，更新 Web 應用程式：

```
gcloud app deploy --quiet
```

`--quiet` 旗標會停用互動式提示，直接啟動部署作業。

應用程式新版本部署完成：

```
Beginning deployment of service [default]...
Uploading 1 file to Google Cloud Storage
...
Deployed service [default] to [https://PROJECT_ID.REGION_ID.r.appspot.com]
```

新版本會以透明方式部署，流量也會自動轉送至新版本 (如果部署成功)。

以先前的方式測試新版網頁應用程式：

```
curl https://$APPENGINE_HOSTNAME
```

您應該會得到相同的答案：

```
Hello World!
```

使用選用參數進行測試：

```
curl https://$APPENGINE_HOSTNAME?who=Universe
```

您應該會收到下列答案：

```
Hello Universe!
```

摘要

在這個步驟中，您更新並重新部署了網路應用程式，服務完全沒有中斷。

步驟 9 · 約 0 分鐘

9. 恭喜！

您已學會使用 Python 編寫第一個 App Engine 網頁應用程式！

瞭解詳情

- App Engine 說明文件：<https://cloud.google.com/appengine>
- 如要瞭解如何在 App Engine 上撰寫功能齊全的 Python 應用程式，請參閱本教學課程：
<https://cloud.google.com/appengine/docs/standard/python3/building-app>

授權

這項內容採用的授權為 Creative Commons 姓名標示 2.0 通用授權。
